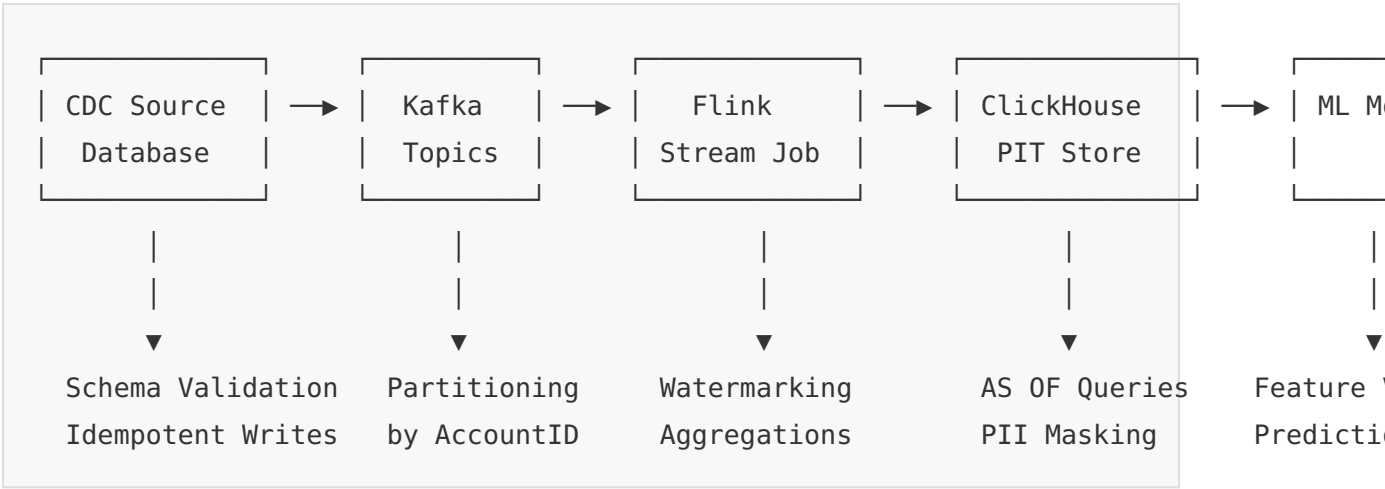


Workflow Guide

This document explains the complete data flow through the PIT Feature Store, from CDC ingestion to ML model consumption.

System Architecture Overview



Data Flow Stages

Stage 1: CDC Ingestion

Source: Database transaction logs (MySQL binlog, PostgreSQL WAL)

Components: - Debezium Connector - Kafka Producer (idempotent) - Schema Registry

Process: 1. Debezium captures database changes in real-time 2. Changes are serialized to Avro/JSON format 3. Schema validation against Schema Registry 4. Idempotent write to Kafka topic with deduplication key 5. Partitioning by `account_id` for ordering guarantees

Key Properties: - Exactly-once semantics enabled - Schema evolution support (backward compatible) - Partition key: `account_id` - Retention: 7 days

Example Event:

```
{
  "event_id": "evt_123456",
  "account_id": "acc_789",
  "event_type": "transaction",
  "event_time": "2024-01-15T10:30:00.123Z",
  "processing_time": "2024-01-15T10:30:00.456Z",
  "payload": {
    "transaction_id": "txn_abc",
    "amount": 150.00,
    "currency": "USD",
    "merchant_category": "retail"
  },
  "schema_version": "1.2"
}
```

Stage 2: Stream Processing (Flink)

Input: Kafka topic `cdc-events`

Components: - Kafka Consumer - Watermark Assigner - KeyBy Operator - Windowed Aggregations - PIT State Manager - ClickHouse Sink

Process: 1. **Consumption:** Read events from Kafka with offset tracking 2. **Watermark Assignment:** Assign watermarks based on `event_time` - Allowed lateness: 5 seconds - Late events routed to side output 3. **KeyBy:** Partition by `account_id` for stateful processing 4. **Windowing:** Tumbling windows (1 minute, 5 minute, 1 hour) 5. **Aggregation:** Compute features per window - Transaction count - Total amount - Average amount - Category distribution 6. **PIT State Update:** Maintain point-in-time correct state - Handle late arrivals with retractions - Version state for historical queries 7. **Sink:** Write to ClickHouse with upsert semantics

Flink DAG:

```
Kafka Source → Watermark → KeyBy(account_id)
      ↓
    Window(1 min) → Aggregate → ProcessFunction
      ↓
    Window(5 min) → Aggregate → ProcessFunction
      ↓
```

Window(1 hour) → Aggregate → ProcessFunction

↓

ClickHouse Sink (upsert)

State Management: - Backend: RocksDB (for large state) - Checkpoint interval: 60 seconds - Mode: EXACTLY_ONCE - Savepoints for upgrades

Stage 3: Storage (ClickHouse)

Tables: 1. `features_raw` - Raw feature events (ReplacingMergeTree) 2. `features_aggregated` - Pre-computed aggregations (AggregatingMergeTree) 3. `feature_history` - Historical snapshots for audit

Schema Design:

```
CREATE TABLE features_raw (  
    account_id String,  
    feature_name String,  
    event_time DateTime64(3),  
    processing_time DateTime64(3),  
    value Float64,  
    version UInt64,  
    is_deleted UInt8  
) ENGINE = ReplacingMergeTree(version)  
PARTITION BY toYYYYMM(event_time)  
ORDER BY (account_id, feature_name, event_time);
```

PIT Query Pattern:

```
SELECT  
    account_id,  
    feature_name,  
    argMax(value, event_time) as pit_value  
FROM features_raw  
WHERE event_time <= '2024-01-15 10:30:00'  
    AND account_id IN ('acc_789', 'acc_790')  
GROUP BY account_id, feature_name;
```

Optimization Techniques: - Projection on `(account_id, feature_name)` - TTL policies for data retention - Sampled reads for large date ranges

Stage 4: Feature Retrieval

Request Pattern:

```
from storage.pit_query_engine import PITQueryEngine

engine = PITQueryEngine(clickhouse_client)

# Get features as of specific timestamp
features = engine.get_features_as_of(
    account_ids=['acc_789', 'acc_790'],
    timestamp='2024-01-15T10:30:00Z',
    feature_names=['tx_count_1m', 'tx_amount_5m']
)

# Apply PII masking
masked_features = pii_tokenizer.mask(features)

# Return to ML model
return masked_features
```

PII Protection: - Tokenization before feature vector assembly - Format-preserving encryption for IDs - Audit log entry for each access

Stage 5: ML Model Consumption

Integration:

```
import mlflow
from pit_feature_store import FeatureStoreClient

client = FeatureStoreClient()

# Get training features (historical)
training_df = client.get_historical_features(
```

```
    entity_ids=account_ids,
    timestamps=event_timestamps,
    feature_list=features
)

# Get serving features (real-time)
serving_features = client.get_online_features(
    entity_ids=[current_account_id],
    feature_list=features
)

# Train model
model = train_model(training_df)
mlflow.log_model(model, "model")

# Serve predictions
predictions = model.predict(serving_features)
```

Security Model

Data Isolation

- Raw CDC data: Encrypted at rest
- PII fields: Tokenized before stream processing
- Feature vectors: Access-controlled via RBAC

Encryption Layers

1. **Transport:** TLS 1.2+ between all components
2. **Storage:** AES-256-GCM for ClickHouse data
3. **PII:** Format-preserving encryption for identifiers

Access Control

```
roles:
  - name: data_engineer
```

```
permissions:
  - read: cdc-events
  - write: features_raw

- name: data_scientist
  permissions:
    - read: feature_vectors
    - no_access: raw_pii

- name: auditor
  permissions:
    - read: audit_logs
    - read: feature_lineage
```

Observability

Metrics Tracked

Metric	Component	Alert Threshold
kafka_consumer_lag	Flink	> 10,000 events
flink_checkpoint_duration	Flink	> 60 seconds
clickhouse_query_latency_p95	ClickHouse	> 100ms
pit_query_latency_p99	Query Engine	> 200ms
pii_masking_failures	Security	> 0

Dashboards

- 1. **Ingestion Dashboard:** Kafka throughput, lag, error rates
- 2. **Stream Processing Dashboard:** Flink checkpoints, backpressure, state size
- 3. **Storage Dashboard:** ClickHouse query latency, merge operations, disk usage
- 4. **Security Dashboard:** PII tokenization success rate, access attempts

Alerting Rules

```
alerts:
  - name: HighConsumerLag
    condition: kafka_consumer_lag > 10000 for 5m
    severity: warning

  - name: CheckpointFailure
    condition: flink_checkpoints_failed > 0
    severity: critical

  - name: PIILeakDetected
    condition: pii_masking_failures > 0
    severity: critical
```

Data Lineage Tracking

Every feature is tracked from source to consumption:

```
Source Table (MySQL)
  ↓ (Debezium CDC)
Kafka Topic (cdc-events)
  ↓ (Flink Stream Job)
Feature Aggregation (windowed)
  ↓ (ClickHouse Sink)
ClickHouse Table (features_raw)
  ↓ (PIT Query)
Feature Vector (timestamp T)
  ↓ (ML Model)
Prediction
```

Lineage Metadata: - Source table and column - Transformation logic (Flink job version) - Aggregation window parameters - Storage location and partition - Query timestamp and parameters - Consumer identity

Error Handling

Late Arrivals

- Events within allowed lateness (5s): Processed normally
- Events beyond allowed lateness: Side output, manual review
- Retractions: Sent to correct historical state

Schema Evolution

- Backward compatible changes: Auto-handled
- Breaking changes: Require job redeployment
- Schema registry validates compatibility

Failure Recovery

1. **Flink Job Failure:** Restore from latest checkpoint
2. **ClickHouse Failure:** Replay from Kafka (retention period)
3. **Kafka Failure:** Debezium buffers until recovery

Performance Optimization

Throughput Optimization

- Kafka: Batch size 16KB, linger.ms 5
- Flink: Parallelism = number of partitions
- ClickHouse: Async inserts, batch size 1000

Latency Optimization

- Watermark alignment: Minimal delay
- Aggregation: Incremental computation
- PIT queries: Pre-computed projections

Cost Optimization

- Data retention: TTL policies (30 days raw, 1 year aggregated)
- Storage tiering: Hot/warm/cold data separation
- Compute scaling: Auto-scaling based on load